FEASIFY SYSTEM ANALYSIS AND ARCHITECTURE SPECIFICATION

Document ID: FEASIFY-DOC-001
Version: 1.0.0
Date: November 10, 2026
Author: Lead QA Orchestrator & Project Discovery Agent
Classification: Academic Capstone Technical Documentation

1. Executive Summary

Feasify is a web-based capstone proposal and feasibility management system designed to support undergraduate and graduate students, faculty advisers, and academic department chairpersons in managing feasibility studies. The system facilitates end-to-end workflows including student group formation, capstone proposal authoring, financial projection computation, artificial intelligence (AI)-assisted feasibility evaluations, official business name verification, and administrative audit logging.

This document presents a comprehensive technical discovery and architectural analysis derived from an empirical inspection of the Feasify repository.

2. Technology Stack Identification

The empirical audit of `package.json`, configuration files, and source trees identifies the actual technology stack:

LAYER	COMPONENT / TOOL	VERSION	PURPOSE
Frontend Framework	React	`^19.2.0`	Declarative component UI library
Frontend Build Tool	Vite	`^7.3.1`	Ultra-fast ESM development and production bundler
Language (Frontend)	TypeScript	`~5.9.3`	Type-safe static analysis for React components
Styling & Design	Tailwind CSS	`^4.2.0`	Utility-first responsive CSS framework
Routing	React Router DOM	`^7.13.0`	Client-side routing and protected route guards
State & Backend as a Service	Firebase Client SDK	`^12.9.0`	Firebase Auth and Cloud Firestore client communication
Icons & UI Primitives	Lucide React	`^0.575.0`	Consistent iconography across dashboards
Mapping & Geolocation	Leaflet	`^1.9.4`	Open-source interactive map engine (OpenStreetMap)
Document Export	jsPDF & jsPDF-AutoTable	`^4.2.1` / `^5.0.9`	Client-side PDF generation of feasibility dossiers
Spreadsheet Processing	SheetJS (xlsx)	`^0.20.1`	Excel file import and export for student rosters
Backend Runtime	Node.js	`v22.20.0` (Active)	Server-side JavaScript runtime
Backend Framework	Express.js	`^5.2.1`	HTTP web application and REST API server
Backend Language	JavaScript (CommonJS)	ES2022	Server-side API logic and service adapters
Backend Admin SDK	Firebase Admin	`^14.3.0`	Server-side privileged Firestore & Auth operations
Real-Time Communication	Socket.io / Socket.io-client	`^4.8.3`	WebSocket bi-directional event notifications
AI Integration	Google Generative AI SDK	`^0.21.0` / `^0.24.1`	Gemini model cascade for financial & proposal analysis
Security & Rate Limiting	express-rate-limit	`^8.6.2`	In-memory IP/Auth token API rate limiting
Email Service	EmailJS Browser SDK	`^4.4.1`	Client-triggered automated email dispatch

3. System Architecture

Feasify utilizes a hybrid client-heavy Single Page Application (SPA) architecture combined with a Node.js/Express companion backend and Firebase cloud services (Auth, Firestore, Cloud Storage).

3.1 Architecture Diagram

```
flowchart TD
    subgraph ClientLayer ["Client Layer (React 19 SPA - Vite)"]
        UI_Student["Student Module\n(/dashboard, /projects, /financial-input, /ai-analysis)"]
        UI_Adviser["Adviser Module\n(/adviser/dashboard, /adviser/airules)"]
        UI_Chair["Chairperson Module\n(/admin/users, /admin/projects)"]
        UI_Common["Shared Components\n(OfficialNameChecker, LocationPickerMap, PDFExport)"]
    end

    subgraph BackendLayer ["Application Server Layer (Node.js & Express 5)"]
        API_Gateway["Express 5 Gateway\n(:10000)"]
        RateLimiter["express-rate-limit\n(60 req/min)"]
        NameCheckerSvc["Name Checker Service\n(DTIProvider & SECProvider)"]
        DynamicExemplars["Dynamic Exemplars Engine\n(Academic Benchmarks)"]
        AI_Cascade["Gemini Model Cascade\n(gemini-2.5-flash → gemini-3.5-flash-lite)"]
        SocketServer["Socket.io Server\n(Real-Time Notification Events)"]
    end

    subgraph DataLayer ["Cloud Persistence Layer (Firebase Cloud)"]
        FB_Auth["Firebase Authentication\n(Email/Password, Token Issuance)"]
        FB_Firestore["Cloud Firestore NoSQL\n(users, groups, proposals, auditLogs)"]
        FB_Storage["Cloud Storage\n(Logos, Attachments)"]
    end

    subgraph ExternalServices ["External Services & Open Data"]
        DTI_Portal["DTI BNRS Portal\n(bnrs.dti.gov.ph)"]
        SEC_eSPARC["SEC eSPARC Portal\n(esparc.sec.gov.ph)"]
        OSM_Tiles["OpenStreetMap / Nominatim\n(Reverse Geocoding Tiles)"]
        Gemini_API["Google Gemini API Cloud\n(LLM Evaluation Engine)"]
    end

    ClientLayer <-->|Firebase Client SDK 12.9| DataLayer
    ClientLayer <-->|HTTP REST / WebSocket| BackendLayer
    BackendLayer -->|Firebase Admin SDK 14.3| DataLayer
    BackendLayer -->|HTTPS API Requests| Gemini_API
    BackendLayer -->|Verification Links & Search| DTI_Portal
    BackendLayer -->|OAuth2 / Verification Links| SEC_eSPARC
    UI_Common -->|Leaflet Tile Requests| OSM_Tiles
```

3.2 Data Flow Architecture

```
sequenceDiagram
    autonumber
    actor Student as Student (Lead Researcher)
    participant ProjectsUI as Projects UI (React)
    participant Backend as Feasify Backend (Express)
    participant Firestore as Cloud Firestore
    actor Adviser as Faculty Adviser

    Student->>ProjectsUI: Input Proposal Title, Tagline, & Category
    ProjectsUI->>Backend: GET /api/teams/:groupId/proposals/count
    Backend-->>ProjectsUI: Return { count: N, allowed: boolean }
    alt count ≥ 3
        ProjectsUI-->>Student: Block creation (3-proposal ceiling reached)
    else count < 3
        Student->>ProjectsUI: Click "Submit Proposal"
        ProjectsUI->>Firestore: addDoc(collection("proposals"), proposalData)
        Firestore-->>ProjectsUI: Proposal Created (docId)
        ProjectsUI->>Firestore: updateDoc(groupRef, { status: "Pending Review" })
        ProjectsUI->>Backend: PUT /api/proposals/:docId/status
        Backend->>Firestore: Update proposal status in Firestore
        Backend-->>Adviser: Emit socket.io event "new_proposal_submitted"
        Adviser->>Firestore: Read submitted proposals for section
    end
```

4. Key Architectural Findings and Discoveries

4.1 Geolocation and Mapping Architecture

The codebase was scrutinized for Google Maps API integration.

- **Finding:** The project does **NOT** use Google Maps API.
- **Actual Implementation:** Feasify implements geolocation through **Leaflet** `^1.9.4` with OpenStreetMap raster tiles, supplemented by an embedded Philippine place search service (`src/services/philippinePlaceSearch.ts`) and offline landmark catalog (`src/data/philippineLandmarks.ts`).
- **Evaluation:** This approach avoids commercial Google Cloud API billing, preventing quota depletion during student usage while providing location picking and clean human-readable address formatting.

4.2 Official Name Checker Architecture

- **DTI BNRS:** Implemented via `feasify-backend/services/nameChecker/dti/DTIProvider.js`. It queries official enterprise API endpoints when credentials are provided. When unconfigured (public tier), it redirects users to the official search link `https://bnrs.dti.gov.ph/search?keyword=...` and clearly returns `NOT_FOUND` without falsely claiming that a name is "officially registered" or "legally available".
- **SEC Philippines:** Implemented via `feasify-backend/services/nameChecker/sec/SECProvider.js`. It explicitly distinguishes corporate entity lookup from definitive name reservation (conducted exclusively via SEC eSPARC).
- **Consolidation:** Company name data is centralized in `feasify-backend/data/companyNames.json` (1,234 records), preventing frontend code bloat.

4.3 Proposal Limit Architecture (Maximum 3 Proposals)

- **Client Enforcement:** `Projects.tsx` enforces `proposals.length ≥ 3` in client-side state and disables the "Submit" and "Save Draft" actions.
- **Backend Endpoint:** `feasify-backend/index.js` provides `GET /api/teams/:groupId/proposals/count` and `POST /api/teams/:groupId/proposals` with strict server-side `count ≥ 3` checks.
- **Critical Finding:** While the backend endpoint exists, `Projects.tsx` writes directly to Cloud Firestore using `addDoc(collection(db, "proposals"))` after querying the count endpoint, creating a potential Time-of-Check to Time-of-Use (TOCTOU) race condition and bypassing backend enforcement if an adversary interacts directly with the Firebase client library.

---

5. Comprehensive Feature Inventory

MODULE NAME	PURPOSE	RELEVANT FILES	ASSOCIATED DB COLLECTIONS	RELATED ENDPOINTS	AUTHORIZED ROLES
Authentication & Profile	User login, password recovery, session verification	`src/Auth.tsx`, `src/ResetPassword.tsx`, `src/firebase.ts`	`users`	Firebase Auth	Public / All
Role-Based Routing	Restricts URL routes by user role	`src/components/ProtectedRoute.tsx`, `src/App.tsx`	`users`	N/A	Student, Adviser, Chairperson
Student Dashboard	High-level metrics, active business summary	`src/Dashboard.tsx`	`groups`, `proposals`	N/A	Student
Proposal Management	Create, edit, draft, and submit capstone proposals (Max 3)	`src/Projects.tsx`	`groups`, `proposals`	`GET/POST /api/teams/:groupId/proposals`	Student (Leader)
Financial Computation	5-year multi-product costing, OPEX, balance sheet	`src/Financial_input.tsx`, `src/utils/productCosting.ts`	`groups`, `proposals`	N/A	Student
AI Feasibility Analysis	Evaluates financial feasibility via Gemini cascade	`src/AI_Analysis.tsx`, `feasify-backend/index.js`	`groups`, `proposals`	`POST /api/analyze`	Student
Official Name Checker	Verifies business/company names vs DTI/SEC	`src/components/OfficialNameChecker.tsx`, `services/nameChecker/`	In-memory cache, `companyNames.json`	`POST /api/name-check`	Student, All
Interactive Map	Location picker with reverse geocoding	`src/components/LocationPickerMap.tsx`	Embedded in `proposals`	OpenStreetMap Nominatim	Student
Adviser Review Dashboard	Review submitted proposals, issue remarks, approve/reject	`src/AdviserDashboard.tsx`	`groups`, `proposals`, `auditLogs`	`PUT /api/proposals/:proposalId/status`	Adviser, Chairperson
Adviser AI Proposal Evaluation	Evaluates student proposals against rubrics	`src/AdviserDashboard.tsx`, `feasify-backend/index.js`	`knowledge_base.json`	`POST /api/analyze-proposal`	Adviser
Chairperson User Management	Add students/advisers, batch import Excel	`src/ChairpersonModule.tsx`	`users`	Firebase Auth	Chairperson
Chairperson Feasibility Overview	Institutional overview of sections and feasibility proposals	`src/ChairpersonFeasib.tsx`	`groups`, `proposals`	N/A	Chairperson
Audit Trail & Logging	Compliance logging of user actions	`src/services/auditLogger.ts`, `feasify-backend/index.js`	`auditLogs`	`GET /api/audit/logs`	Adviser, Chairperson
Dossier PDF Export	Export formatted PDF feasibility study document	`src/utils/exportProposalAnalysisPDF.ts`, `src/Reports.tsx`	`proposals`	N/A (Client jsPDF)	Student, Adviser